Assignment = 2

Name = Karan

Section = AE (36)

University Roll no. = 2415000772

Subject = Computer Organization

Subject code = (BCSC – 1005)

Q1. Pipeline configuration for evaluating Ai × Bi × (Ci × Di)

We must compute:

$$\text{Result}_i = A_i \times B_i \times (C_i \times D_i)$$

This requires **three multiplications**. A suitable pipeline is:

Pipeline Stages

1. **Stage S1 – Fetch operands**: Fetch Ai , Bi , Ci , Di
2. **Stage S2 – Multiply M1**: Compute P1 = Ai × Bi
3. **Stage S3 – Multiply M2**: Compute P2 = Ci × Di
4. **Stage S4 – Multiply M3**: Compute Result = P1 × P2

Diagram:

Stage	Operation
S1	Fetch Ai, Bi, Ci, Di
S2	P1 = Ai × Bi
S3	P2 = Ci × Di
S4	Result = P1 × P2

Q2. Four-stage pipeline timing

Given

Stage delays: 150 ns, 120 ns, 160 ns, 140 ns
Register delay = 5 ns

Clock cycle time

$$T_{clk} = \max(\text{stage delays}) + \text{register delay}$$
$$T_{clk} = 160 + 5 = 165 \text{ ns}$$

Time for 1000 items

Pipeline time:

$$T = (k + n - 1)T_{clk}$$

k = 4 stages, n = 1000 inputs

$$T = (4 + 1000 - 1) \times 165 = 1003 \times 165 = 165495 \text{ ns}$$

Speedup

Non-pipeline time per item = sum of stage delays

$$= 150 + 120 + 160 + 140 = 570 \text{ ns}$$

Time for 1000 items without pipeline:

$$1000 \times 570 = 570000 \text{ ns}$$

$$\text{Speedup} = \frac{570000}{165495} \approx 3.44$$

Throughput

$$\text{Throughput} = \frac{1}{T_{clk}} = \frac{1}{165 \text{ ns}} \approx 6.06 \times 10^6 \text{ items/sec}$$

Q3. Two-way set associative cache

Given

- Block size = 4 words
- Cache size = 2048 words
- Cache is **2-way set associative**
- Main memory size = 128K × 32 (128K words)

(i) Required fields

Number of blocks in cache

$$\frac{2048 \text{ words}}{4 \text{ words/block}} = 512 \text{ blocks}$$

Since 2-way:

$$\text{No. of sets} = \frac{512}{2} = 256 \text{ sets}$$

Address fields

Main memory size = 128K = 2^17 words → 17-bit address.

- **Word offset** = $\log_2 4$ = 2 bits
- **Index** = $\log_2 256$ = 8 bits
- **Tag** = 17 – 8 – 2 = 7 bits

(ii) Size of cache

Cache data size = 2048 words × 32 bits = 65536 bits
Plus tag + valid bits etc.

Q4. Virtual memory, FIFO & LRU (Page size 1k)

Address space = 8 pages
Memory space = 4 pages
Reference string:
4, 2, 0, 1, 2, 6, 1, 4, 0, 1, 0, 2, 3, 5, 7

(i) FIFO Page Replacement

Simulate:

Ref	Frames (4 max)
4	4
2	4 2
0	4 2 0
1	4 2 0 1
2	Hit → same

Ref	Frames (4 max)
6	2 0 1 6
1	Hit
4	0 1 6 4
0	Hit
1	Hit
0	Hit
2	1 6 4 2
3	6 4 2 3
5	4 2 3 5
7	2 3 5 7

(ii) LRU

You may request full table if you want—it is long but doable.

Final frames:
→ **FIFO:** 2,3,5,7
→ **LRU:** 3,5,7,2

Q5. Cache access time

Given

Cache=120ns
Mainmemory=900ns

Read=80%(hitratio=0.9)
Write = 20% (write-through)

(i) Average read access time

Write-through does NOT affect read cycles:

$$T_{read} = 0.9(120) + 0.1(900) = 108 + 90 = 198 \text{ ns}$$

(ii) Hit ratio including write cycle

Write-through: write goes to cache + main memory.

Write hit rate = irrelevant → writes always go to memory.

Effective hit ratio:

$$H = 0.8(0.9) = 0.72$$

(iii) Average access time (read + write)

Write access:

$$T_{write} = 900 \text{ ns}$$

Total:

$$T = 0.8(198) + 0.2(900)$$
$$T = 158.4 + 180 = 338.4 \text{ ns}$$

Q6. RAM chips 1024×1 needed

(a) To build 1024 bytes = 1024×8 bits

Eachchip=1024×1→1Kbit
Need 8 chips (1 for each bit).

Address lines: 10 lines (because 1024 = 2^{10}) connected in **parallel** to all chips.

(b) To build 16K bytes = 16K×8 bits = 128K bits

Eachchip:1Kbits→
Number of chips:

$$\frac{128K}{1K} = 128 \text{ chips}$$

Connected as:

- 8 chips form 1 byte
- Need 16K bytes → 16K rows → requires **address decoding** to select a group of 8 chips at a time.

Q7. Micro-operations

(i) STA (Store AC to memory)

MAR←IR(address)
MDR←AC
Write
PC ← PC + 1

(ii) ADD to AC

MAR←IR(address)
Read

AC←AC+MDR
PC ← PC + 1

(iii) CIR (Circular shift right)

AC = AC >> 1 with AC(0) ← AC(15)

(iv) INC (increment AC)

AC ← AC + 1

Q8. Virtual memory page faults

Pages in memory: {0,1,4,6}

Virtual pages = 0–7

Pages causing page faults are the others that's because of the address: → **2, 3, 5, 7**

All addresses in these page ranges cause faults:

- Page 2 → 2048–3071
- Page 3 → 3072–4095
- Page 5 → 5120–6143
- Page 7 → 7168–8191

Q9. Basic Computer ADD instruction

Given:
PC=7FF
M[7FF]=9A9F
Thus IR = 9A9F

Address field=A9F
M[A9F]=0C35
AC = AC + MDR → AC = AC + 0C35

Final:
PC=800
AR=A9F
DR=0C35
AC = previous AC + 0C35

Q10. Write policies in cache

(i) Write-through

- Update cache + memory simultaneously
  Pros: simple, memory always updated
  Cons: slow due to frequent memory writes

(ii) Write-back

- Update only cache; write to memory on replacement
  Pros:fast
  Cons: dirty-bit needed, memory inconsistent until write-back

(iii) Write-around

- Writes go directly to memory (cache not updated)
  Pros: avoids cache pollution
  Cons: slows writes

Q11. Asynchronous transfer

Strobe method

A single control signal (Strobe) indicates valid data. Requires timing agreement.

Handshaking

Two signals:

- **Data Valid**
- **Acknowledge**

Sender waits until receiver acknowledges.

Q12. Cache fields

Cache=4Kwords
Block size=64words
4 blocks per set

Number of blocks in cache = 4096/64 = 64
Sets = 64 / 4 = 16 sets

Fields

Word field=$\log_2 64$=6bits
Set field = $\log_2 16$ = 4 bits

Q13. DMA read/write lines are bidirectional

- When CPU sends command → lines act as input to DMA
- When DMA transfers data to memory → act as output
  Used for DMA read/write cycles.

Q14. (Repeated) Write policies

Already answered in Q10.

Q15. Hardwired vs Microprogrammed Control Unit

Processor A (real-time, fast)

→ **Hardwired Control**

- Faster execution
- Optimized circuitry
  Suitable for real-time systems.

Processor B (general purpose, modifiable)

→ **Microprogrammed Control**

- Easy to modify instructions
- Supports complex instruction sets
  Suitable for general-purpose designs.

Q16. Increment AC instruction

PC = 3BF
M[3BF] = 7020 → this is INC AC instruction

After fetch:
MAR = 3BF
MDR = 7020
IR = 7020
PC = 3C0

Execution:
AC = DCF9 + 1 = DCFA

Final:

- **PC = 3C0**
- **MAR = 3BF**
- **MDR = 7020**
- **AC = DCFA**